

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Facultad de Ciencias Puras
Carrera de Ingeniería Informática

GUÍA DE LABORATORIO N.º 8

Autenticación con JWT y Gestión Segura de Sesiones

Access / Refresh · Rotación · Revocación · Cookies httpOnly

NestJS 11 · PostgreSQL · TypeORM

Asignatura	Seguridad de Software
Sigla	INF-781
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Unidad temática	Autenticación y gestión de sesiones
Proyecto	inf781-jwt-sessions (desde cero)
Gestión	I - 2026

1. Objetivos

Objetivo general

Construir desde cero una API de autenticación con NestJS y PostgreSQL que emplee access tokens y refresh tokens, con persistencia y rotación del refresh, revocación real, detección de reuso y entrega mediante cookies httpOnly, soportando sesiones en múltiples dispositivos.

Objetivos específicos

- Crear un proyecto NestJS nuevo y conectarlo a PostgreSQL mediante TypeORM.
- Implementar registro e inicio de sesión con hashing seguro de contraseñas (argon2).
- Emitir y verificar tokens con @nestjs/jwt y passport-jwt a través de estrategias y guards.
- Persistir un hash del refresh token con una fila por sesión/dispositivo.
- Aplicar rotación e implementar la detección de reuso de tokens rotados.
- Habilitar logout por dispositivo, revocación global y listado de sesiones activas.
- Entregar el refresh token en una cookie httpOnly + Secure + SameSite.

2. Competencia de la unidad

El estudiante diseña e implementa un mecanismo completo de autenticación basado en tokens para servicios web, aplicando buenas prácticas en el manejo de credenciales y evaluando los vectores de ataque asociados al almacenamiento y la renovación de tokens.

3. Marco teórico

3.1 ¿Qué es un JWT?

Un **JSON Web Token (JWT)**, definido en el RFC 7519, es un objeto JSON compacto y firmado digitalmente que permite transmitir información entre dos partes de forma verificable. Se compone de tres partes en Base64URL separadas por puntos: **header.payload.signature**. El header indica el algoritmo (p. ej. HS256), el payload contiene los claims y la firma garantiza integridad y autenticidad.

Firmado no es cifrado

El payload está codificado, no cifrado: cualquiera puede leerlo decodificando el Base64URL. La firma solo impide modificarlo. Por eso nunca se colocan datos sensibles dentro de un JWT.

3.2 Access token y refresh token

Para equilibrar seguridad y comodidad se usan dos tokens: uno de acceso, de vida corta, y uno de renovación, de vida larga.

Característica	Access Token	Refresh Token
Duración	Corta (p. ej. 15 min)	Larga (p. ej. 7 días)
Uso	Acceder a rutas protegidas	Obtener nuevos access tokens
Se envía en	Header Authorization Bearer	Cookie httpOnly (solo a /auth)
Riesgo si se filtra	Limitado (caduca pronto)	Alto (vida larga)

3.3 Por qué persistir y rotar el refresh token

Si el servidor solo confiara en la firma del refresh (modelo puramente stateless), no podría revocar una sesión antes de su expiración y un token robado serviría hasta caducar. Para evitarlo guardamos en la base de datos un hash del refresh token por cada sesión. Esto habilita tres capacidades:

- **Revocación.** El logout marca la sesión como revocada en el servidor.
- **Rotación.** Cada renovación emite un refresh nuevo e invalida el anterior.
- **Detección de reúso.** Si se presenta un refresh ya rotado, se asume robo y se revocan todas las sesiones del usuario.

Una sesión = un dispositivo

Cada inicio de sesión crea una fila en la tabla refresh_tokens con un sessionId único que viaja como claim dentro del refresh JWT. Así el usuario puede mantener sesiones simultáneas en varios dispositivos, listarlas y cerrarlas de forma individual.

3.4 Cookies httpOnly frente a localStorage

Aspecto	localStorage	Cookie httpOnly
Acceso desde JavaScript	Sí (vulnerable a XSS)	No (protegido de XSS)
Envío al servidor	Manual (header)	Automático por el navegador
Riesgo principal	XSS	CSRF (se mitiga con SameSite)

Guardar el refresh en una cookie **httpOnly** lo hace inaccesible a JavaScript y elimina el riesgo de XSS, a cambio de introducir el de CSRF, que mitigamos con **SameSite=strict** y restringiendo el path de la cookie a las rutas de autenticación.

4. Requisitos previos

- **Node.js** 20 LTS o superior y npm.
- **PostgreSQL** 14 o superior en ejecución, con un usuario y una base de datos disponibles.
- **NestJS CLI** instalado globalmente (npm i -g @nestjs/cli).
- **Un cliente HTTP que gestione cookies:** Postman, Bruno o REST Client de VS Code.

- **Conocimientos previos:** TypeScript, módulos y proveedores de NestJS, y hashing de contraseñas.

Versión de referencia

Usaremos NestJS 11 (estable, sobre CommonJS), la versión recomendada para el semestre I-2026. NestJS 12, previsto para el tercer trimestre de 2026, migrará a ESM; no lo usaremos aquí para evitar incompatibilidades con el tooling actual.

5. Desarrollo paso a paso

Construiremos la API de forma incremental. Primero la base (proyecto, conexión, usuarios y login con JWT) y luego las capacidades de seguridad (persistencia, rotación, reuso, cookies). La estructura final será:

```
src/
├── app.module.ts
├── main.ts
├── users/
│   ├── user.entity.ts
│   ├── users.module.ts
│   └── users.service.ts
├── auth/
│   ├── auth.module.ts
│   ├── auth.service.ts
│   ├── auth.controller.ts
│   ├── refresh-token.entity.ts
│   ├── dto/ (register.dto.ts, login.dto.ts)
│   ├── strategies/ (access-token, refresh-token)
│   ├── guards/ (access-token, refresh-token)
│   └── decorators/ (get-current-user)
```

Paso 1 — Crear el proyecto

Generamos un proyecto NestJS nuevo y entramos en la carpeta. Al ejecutarlo, elige npm como gestor de paquetes.

```
nest new inf781-jwt-sessions
cd inf781-jwt-sessions
```

Paso 2 — Instalar dependencias

Instalamos cuatro grupos de paquetes: TypeORM y el driver de PostgreSQL; JWT y Passport; el hasher y los validadores; y cookie-parser para leer cookies.

```
npm install @nestjs/typeorm typeorm pg \
  @nestjs/jwt @nestjs/passport passport passport-jwt \
  @nestjs/config argon2 class-validator class-transformer \
  cookie-parser

npm install -D @types/passport-jwt @types/cookie-parser
```

Paso 3 — Variables de entorno

Creamos un archivo `.env` en la raíz. Los secretos de access y refresh deben ser largos, aleatorios y distintos entre sí. Recuerda añadir `.env` a tu `.gitignore`.

Archivo: `.env`

```
NODE_ENV=development
CLIENT_ORIGIN=http://localhost:5173

DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USER=jwt_sessions_user
DATABASE_PASSWORD=jwt_sessions_password
DATABASE_NAME=jwt_sessions_db

JWT_ACCESS_SECRET=mi-mecreto
JWT_REFRESH_SECRET=mi-secreto-refresh
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
```

Paso 4 — Conexión a PostgreSQL (módulo raíz)

En `app.module.ts` cargamos `ConfigModule` de forma global y configuramos la conexión de `TypeORM`. La opción `synchronize` crea las tablas automáticamente; es cómoda para aprender, pero solo debe usarse en desarrollo.

Archivo: `src/app.module.ts`

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../users/user.entity';
import { RefreshToken } from '../auth/refresh-token.entity';
import { UsersModule } from '../users/users.module';
import { AuthModule } from '../auth/auth.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: process.env.DATABASE_HOST,
      port: parseInt(process.env.DATABASE_PORT, 10),
      username: process.env.DATABASE_USER,
      password: process.env.DATABASE_PASSWORD,
      database: process.env.DATABASE_NAME,
      entities: [User, RefreshToken],
      synchronize: true,
    }),
    // Falta los modulos
    UsersModule,
  ],
})
```

```
    AuthModule,  
  ],  
})  
export class AppModule {}
```

Paso 5 — Entidad User

Define al usuario. El campo password almacena únicamente el hash, nunca el texto plano.

Archivo: *src/users/user.entity.ts*

```
import {  
  Column,  
  CreateDateColumn,  
  Entity,  
  PrimaryGeneratedColumn,  
} from 'typeorm';  
  
@Entity('users')  
export class User {  
  @PrimaryGeneratedColumn('uuid')  
  id: string;  
  
  @Column({ unique: true })  
  email: string;  
  
  @Column()  
  password: string;  
  
  @CreateDateColumn()  
  createdAt: Date;  
}
```

Paso 6 — Entidad RefreshToken (una fila por sesión)

Cada sesión activa es una fila. Guardamos el hash del refresh token, el dispositivo (userAgent), un indicador de revocación y la fecha de expiración. La relación con User usa borrado en cascada.

Archivo: *src/auth/refresh-token.entity.ts*

```
import { User } from 'src/users/user.entity';  
import {  
  Column,  
  CreateDateColumn,  
  Entity,  
  JoinColumn,  
  ManyToOne,  
  PrimaryColumn,  
}
```

```
} from 'typeorm';

@Entity('refresh_tokens')
export class RefreshToken {
  @PrimaryColumn('uuid')
  id: string;

  @Column()
  userId: string;

  @ManyToOne(() => User, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'userId' })
  user: User;

  @Column()
  hashedToken: string;

  @Column({ type: 'varchar', nullable: true })
  userAgent: string | null;

  @Column({ default: false })
  revoked: boolean;

  @Column({ type: 'timestampz' })
  expiresAt: Date;

  @CreateDateColumn()
  createdAt: Date;
}
```

Paso 7 — Servicio y módulo de usuarios

El servicio de usuarios encapsula el acceso a la tabla users. Exportamos UsersService para que AuthModule pueda inyectarlo.

Archivo: `src/users/users.service.ts`

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { Repository } from 'typeorm';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly repo: Repository<User>,
  ) {}
}
```



```
create(email: string, password: string): Promise<User> {
  return this.repo.save(this.repo.create({ email, password }));
}

findByEmail(email: string): Promise<User | null> {
  return this.repo.findOne({ where: { email } });
}
}
```

Archivo: *src/users/users.module.ts*

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { UsersService } from './users.service';

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}
```

Paso 8 — DTOs con validación

Los DTOs validan la entrada con `class-validator`. El `ValidationPipe` global (Paso 17) los aplica automáticamente.

Archivo: *src/auth/dto/register.dto.ts*

```
import { IsEmail, IsString, MinLength } from 'class-validator';

export class RegisterDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  password: string;
}
```

Archivo: *src/auth/dto/login.dto.ts*

```
import { IsEmail, IsString, MinLength } from 'class-validator';

export class LoginDto {
  @IsEmail()
  email: string;
}
```

```
@IsString()
@MinLength(8)
password: string;
}
```

Paso 9 — AuthService: registro, login y emisión de sesión

El servicio hashea contraseñas con argon2 y verifica credenciales. `issueSession()` genera un `sessionId`, firma el par de tokens (incluyendo el `sessionId` como claim del refresh) y persiste el hash del refresh en la tabla.

Archivo: `src/auth/auth.service.ts`

```
import {
  ForbiddenException,
  Injectable,
  UnauthorizedException,
} from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { JwtService } from '@nestjs/jwt';
import { InjectRepository } from '@nestjs/typeorm';
import { UsersService } from 'src/users/users.service';
import { RefreshToken } from './refresh-token.entity';
import { Repository } from 'typeorm';
import { RegisterDto } from './dto/register.dto';
import * as argon2 from 'argon2';
import { randomUUID } from 'crypto';
import { LoginDto } from './dto/login.dto';

@Injectable()
export class AuthService {
  constructor(
    private readonly users: UsersService,
    private readonly jwt: JwtService,
    private readonly config: ConfigService,
    @InjectRepository(RefreshToken)
    private readonly tokens: Repository<RefreshToken>,
  ) {}

  async register(dto: RegisterDto, ua?: string) {
    if (await this.users.findByEmail(dto.email))
      throw new ForbiddenException('El correo ya está registrado.');
```

```
    const hash = await argon2.hash(dto.password);
    const user = await this.users.create(dto.email, hash);
    return this.issueSession(user.id, user.email, ua);
  }

  private async issueSession(userId: string, email: string, ua?: string) {
    const sessionId = randomUUID();
```

```
const pair = await this.signTokens(userId, email, sessionId);
await this.persist(sessionId, userId, pair.refreshToken, ua);
return pair;
}

private async signTokens(sub: string, email: string, sessionId: string) {
  const [accessToken, refreshToken] = await Promise.all([
    this.jwt.signAsync(
      { sub, email, sessionId },
      {
        secret: this.config.get('JWT_ACCESS_SECRET'),
        expiresIn: this.config.get('JWT_ACCESS_EXPIRES'),
      },
    ),
    this.jwt.signAsync(
      { sub, email, sessionId },
      {
        secret: this.config.get('JWT_REFRESH_SECRET'),
        expiresIn: this.config.get('JWT_REFRESH_EXPIRES'),
      },
    ),
  ]);
  return { accessToken, refreshToken };
}

private async persist(
  id: string,
  userId: string,
  token: string,
  ua?: string,
) {
  await this.tokens.save(
    this.tokens.create({
      id,
      userId,
      hashedToken: await argon2.hash(token),
      userAgent: ua ?? null,
      revoked: false,
      expiresAt: this.refreshExpiry(),
    }),
  );
}

private refreshExpiry(): Date {
  const d = new Date();
  d.setDate(d.getDate() + 7);
  return d;
}
```

```
}

async login(dto: LoginDto, ua?: string) {
  const user = await this.users.findByEmail(dto.email);
  if (!user) throw new UnauthorizedException('Credenciales inválidas');
  const ok = await argon2.verify(user.password, dto.password);
  if (!ok) throw new UnauthorizedException('Credenciales inválidas');
  return this.issueSession(user.id, user.email, ua);
}
}
```

Paso 10 — Rotación y detección de reúso

Añadimos `refreshTokens()` al servicio. Busca la sesión por su `sessionId`; si no existe, está revocada o caducó, revoca todo. Luego compara el token presentado con el hash guardado: si no coincide, es un token rotado reutilizado y también revoca todo. Si todo es correcto, rota emitiendo un par nuevo y sobrescribiendo el hash.

Archivo: `src/auth/auth.service.ts` (dentro de `AuthService`)

```
async refreshTokens(
  userId: string,
  email: string,
  sessionId: string,
  presented: string,
  ua?: string,
) {
  const session = await this.tokens.findOne({ where: { id: sessionId } });

  if (!session || session.revoked || session.expiresAt < new Date()) {
    await this.revokeAll(userId);
    throw new ForbiddenException('Sesión inválida. Inicia sesión de nuevo.');
```

```
}
```

```
const ok = await argon2.verify(session.hashedToken, presented);
if (!ok) {
  await this.revokeAll(userId); // reúso de token rotado
  throw new ForbiddenException(
    'Reúso detectado. Se cerraron todas las sesiones.',
  );
}
```

```
const pair = await this.signTokens(userId, email, sessionId);
session.hashedToken = await argon2.hash(pair.refreshToken);
session.expiresAt = this.refreshExpiry();
session.userAgent = ua ?? session.userAgent;
await this.tokens.save(session);
return pair;
```

```
}
```

Paso 11 — Logout, revocación global y listado de sesiones

Archivo: `src/auth/auth.service.ts` (dentro de `AuthService`)

```
async logout(sessionId: string) {
  await this.tokens.update({ id: sessionId }, { revoked: true });
}

async revokeAll(userId: string) {
  await this.tokens.update({ userId, revoked: false }, { revoked: true });
}

listSessions(userId: string) {
  return this.tokens.find({
    where: { userId, revoked: false },
    select: {
      id: true,
      userAgent: true,
      createdAt: true,
      expiresAt: true,
    },
  });
}
```

Paso 12 — Estrategia del access token

Extrae el access token del header Bearer y valida su firma con el secreto de acceso. Lo que devuelve `validate()` se adjunta a `request.user`.

Archivo: `src/auth/strategies/access-token.strategy.ts`

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { PassportStrategy } from '@nestjs/passport';
import { InjectRepository } from '@nestjs/typeorm';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { Repository } from 'typeorm';
import { RefreshToken } from '../refresh-token.entity';

type Payload = { sub: string; email: string; sessionId: string };

@Injectable()
export class AccessTokenStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(
```

```
config: ConfigService,  
@InjectRepository(RefreshToken)  
private readonly tokens: Repository<RefreshToken>,  
) {  
  super({  
    jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),  
    secretOrKey: config.get('JWT_ACCESS_SECRET'),  
  });  
}  
  
async validate(payload: Payload) {  
  if (!payload?.sessionId)  
    throw new UnauthorizedException('Sesión inválida.');  const session = await this.tokens.findOne({  
    where: { id: payload.sessionId },  
  });  
  if (!session || session.revoked)  
    throw new UnauthorizedException('Sesión inválida.');  return payload;  
}
```

Paso 13 — Estrategia del refresh token (desde la cookie)

A diferencia del access token, el refresh se lee de la cookie mediante un extractor personalizado. `passReqToCallback` nos permite recuperar su valor crudo para compararlo con el hash almacenado.

Archivo: `src/auth/strategies/refresh-token.strategy.ts`

```
import { Injectable } from '@nestjs/common';  
import { ConfigService } from '@nestjs/config';  
import { PassportStrategy } from '@nestjs/passport';  
import { ExtractJwt, Strategy } from 'passport-jwt';  
import { Request } from 'express';  
  
type Payload = { sub: string; email: string; sessionId: string };  
  
const fromCookie = (req: Request): string | null =>  
  req?.cookies?.refreshToken ?? null;  
  
@Injectable()  
export class RefreshTokenStrategy extends PassportStrategy(  
  Strategy,  
  'jwt-refresh',  
) {  
  constructor(config: ConfigService) {  
    super({  
      jwtFromRequest: ExtractJwt.fromExtractors([fromCookie]),  

```

```
    secretOrKey: config.get('JWT_REFRESH_SECRET'),
    passReqToCallback: true,
  });
}
validate(req: Request, payload: Payload) {
  return { ...payload, refreshToken: req?.cookies?.refreshToken };
}
}
```

Paso 14 — Guards y decorador

Los guards aplican cada estrategia a las rutas. El decorador `@GetCurrentUser` facilita leer el usuario autenticado o uno de sus campos.

Archivo: `src/auth/guards/access-token.guard.ts`

```
// guards/access-token.guard.ts
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
@Injectable()
export class AccessTokenGuard extends AuthGuard('jwt') {}
```

Archivo: `src/auth/guards/refresh-token.guard.ts`

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
@Injectable()
export class RefreshTokenGuard extends AuthGuard('jwt-refresh') {}
```

Archivo: `src/auth/decorators/get-current-user.decorator.ts`

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export const GetCurrentUser = createParamDecorator(
  (data: string | undefined, ctx: ExecutionContext) => {
    const req = ctx.switchToHttp().getRequest();
    return data ? req.user[data] : req.user;
  },
);
```

Paso 15 — AuthModule

Registra la entidad `RefreshToken`, `Passport`, `JwtModule` (vacío: los secretos se pasan al firmar) y las dos estrategias como providers.

Archivo: `src/auth/auth.module.ts`

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { TypeOrmModule } from '@nestjs/typeorm';
```

```
import { UsersModule } from '../users/users.module';
import { RefreshToken } from './refresh-token.entity';
import { AuthService } from './auth.service';
import { AccessTokenStrategy } from './strategies/access-token.strategy';
import { RefreshTokenStrategy } from './strategies/refresh-token.strategy';
import { AuthController } from './auth.controller';

@Module({
  imports: [
    PassportModule,
    JwtModule.register({}),
    UsersModule,
    TypeOrmModule.forFeature([RefreshToken]),
  ],
  controllers: [AuthController],
  providers: [AuthService, AccessTokenStrategy, RefreshTokenStrategy],
})
export class AuthModule {}
```

Paso 16 — AuthController con cookies httpOnly

El controlador escribe el refresh en una cookie httpOnly y devuelve solo el accessToken en el cuerpo. El logout usa el guard de refresh porque necesita el sessionId de la cookie.

Archivo: `src/auth/auth.controller.ts`

```
import {
  Body,
  Controller,
  Get,
  HttpStatusCode,
  HttpStatus,
  Post,
  Req,
  Res,
  UseGuards,
} from '@nestjs/common';
import { Response, Request } from 'express';
import { AuthService } from './auth.service';
import { RegisterDto } from './dto/register.dto';
import { LoginDto } from './dto/login.dto';
import { AccessTokenGuard } from './guards/access-token.guard';
import { RefreshTokenGuard } from './guards/refresh-token.guard';
import { GetCurrentUser } from './decorators/get-current-user.decorator';

@Controller('auth')
export class AuthController {
  constructor(private readonly auth: AuthService) {}
```



```
private setRefreshCookie(res: Response, token: string) {
  res.cookie('refreshToken', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    path: '/auth',
    maxAge: 7 * 24 * 60 * 60 * 1000,
  });
}

@Post('register')
async register(
  @Body() dto: RegisterDto,
  @Req() req: Request,
  @Res({ passthrough: true }) res: Response,
) {
  const p = await this.auth.register(dto, req.headers['user-agent']);
  this.setRefreshCookie(res, p.refreshToken);
  return { accessToken: p.accessToken };
}

@HttpCode(HttpStatus.OK)
@Post('login')
async login(
  @Body() dto: LoginDto,
  @Req() req: Request,
  @Res({ passthrough: true }) res: Response,
) {
  const p = await this.auth.login(dto, req.headers['user-agent']);
  this.setRefreshCookie(res, p.refreshToken);
  return { accessToken: p.accessToken };
}

@UseGuards(RefreshTokenGuard)
@HttpCode(HttpStatus.OK)
@Post('refresh')
async refresh(
  @GetCurrentUser() u: any,
  @Req() req: Request,
  @Res({ passthrough: true }) res: Response,
) {
  const p = await this.auth.refreshTokens(
    u.sub,
    u.email,
    u.sessionId,
    u.refreshToken,
    req.headers['user-agent'],
  );
}
```

```

    );
    this.setRefreshCookie(res, p.refreshToken);
    return { accessToken: p.accessToken };
}

@UseGuards(RefreshTokenGuard)
@HttpCode(HttpStatus.OK)
@Post('logout')
async logout(
  @GetCurrentUser('sessionId') sessionId: string,
  @Res({ passthrough: true }) res: Response,
) {
  await this.auth.logout(sessionId);
  res.clearCookie('refreshToken', { path: '/auth' });
  return { message: 'Sesión cerrada' };
}

@UseGuards(AccessTokenGuard)
@Get('me')
me(@GetCurrentUser() user: { sub: string; email: string }) {
  return user;
}

@UseGuards(AccessTokenGuard)
@Get('sessions')
sessions(@GetCurrentUser('sub') userId: string) {
  return this.auth.listSessions(userId);
}
}

```

Paso 17 — main.ts: cookie-parser, CORS y validación

Activamos cookie-parser para leer cookies, CORS con credenciales para que el navegador envíe la cookie, y el ValidationPipe global para validar los DTOs.

Archivo: src/main.ts

```

import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import * as cookieParser from 'cookie-parser';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(cookieParser());
  app.enableCors({ origin: process.env.CLIENT_ORIGIN, credentials: true });
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,

```

```
      forbidNonWhitelisted: true,
    }),
  );
  await app.listen(3000);
}
bootstrap();
```

Paso 18 — Probar el flujo completo

Arranca con `npm run start:dev`. El cliente guarda la cookie automáticamente. Ejecuta en orden:

Base URL: `http://localhost:3000`

1. POST /auth/register — Registrar usuario

- **Body** → raw → JSON:

```
{
  "email": "usuario@test.com",
  "password": "mipassword123"
}
```

- **Respuesta:** { "accessToken": "eyJ..." }
- El refreshToken llega automáticamente como **cookie** (httpOnly).

2. POST /auth/login — Iniciar sesión

- **Body** → raw → JSON:

```
{
  "email": "usuario@test.com",
  "password": "mipassword123"
}
```

- **Respuesta:** { "accessToken": "eyJ..." }
- El refreshToken llega como **cookie**.

3. GET /auth/me — Datos del usuario actual (protegido)

- **Authorization** → Bearer Token → pega el accessToken
- **Respuesta:** { "sub": "...", "email": "..." }

4. GET /auth/sessions — Listar sesiones activas (protegido)

- **Authorization** → Bearer Token → pega el accessToken

5. POST /auth/refresh — Renovar access token

- No necesita body ni Authorization header.
- Postman debe enviar la **cookie** refreshToken automáticamente (asegúrate de que "Send cookies" esté activo en Postman).

6. POST /auth/logout — Cerrar sesión

- Requiere la **cookie** refreshToken (igual que refresh).
- **Respuesta:** { "message": "Sesión cerrada" }

6. Entregables

- Repositorio Git con el proyecto inf781-jwt-sessions funcional (con .env excluido en .gitignore) y un README de ejecución.
- Capturas de pantalla de las pruebas.

7. Rúbrica de evaluación

Criterio	Descripción	Puntos
Configuración	Proyecto NestJS + PostgreSQL y entidades User y RefreshToken correctas.	15
Registro y login	Hashing con argon2 y verificación de credenciales.	15
Tokens y persistencia	Emisión de access/refresh y almacenamiento del hash por sesión.	20
Rotación y reúso	Rotación correcta y revocación global ante reúso detectado.	20
Cookies y revocación	Cookie httpOnly, logout por dispositivo y listado de sesiones.	20
Análisis y actividades	Comprensión de los riesgos y resolución de actividades.	10
TOTAL		100

8. Bibliografía y referencias

NestJS. (2026). *Authentication & Authorization. Documentación oficial* — docs.nestjs.com/security/authentication

OWASP. *Cheat Sheet Series — JSON Web Token & Session Management*.

Auth0. *Refresh Token Rotation and Automatic Reuse Detection*.

MDN Web Docs. *Set-Cookie: HttpOnly, Secure y SameSite*.

Internet Engineering Task Force. (2015). *RFC 7519: JSON Web Token (JWT)*.